

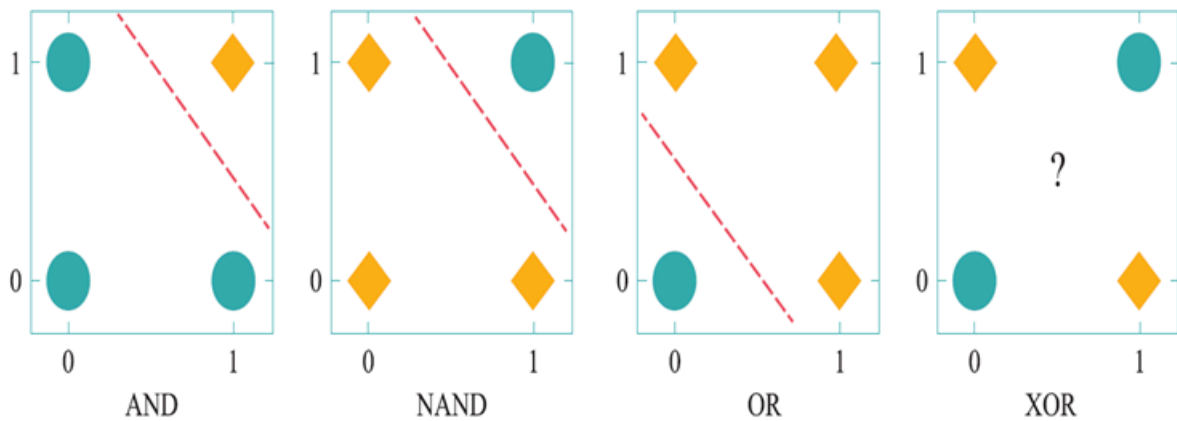


深度学习

神经网络

感知机

- 仅包含输入层和输出层的两层神经网络
- 可完成两类问题的线性分类
- 无法完成逻辑异或（XOR）这一非线性可分逻辑函数任务
- 增加隐藏层会因计算量过大而无法学习



多层感知机

- 反向误差传播算法解决参数优化难题
- 隐藏层中每个神经元均包含具备非线性映射能力的激活函数(如sigmoid)，因此其可用来构造复杂的非线性分类函数

深度信念网络

- 预训练：为神经网络中的连接权重找到一个接近最优解的值
- 微调：对整个网络的参数进行优化训练
- 减少了训练多层神经网络的时间开销
- 命名为深度学习

前馈神经网络

输入层、隐藏层、输出层，每层神经元只和相邻层相连，向后序层传播

MCP神经元模型

- 输入n个0或1的数据
- 给定连接参数
- 对数据**线性加权求和**并通过**激活函数**映射为0或1以完成二分类任务

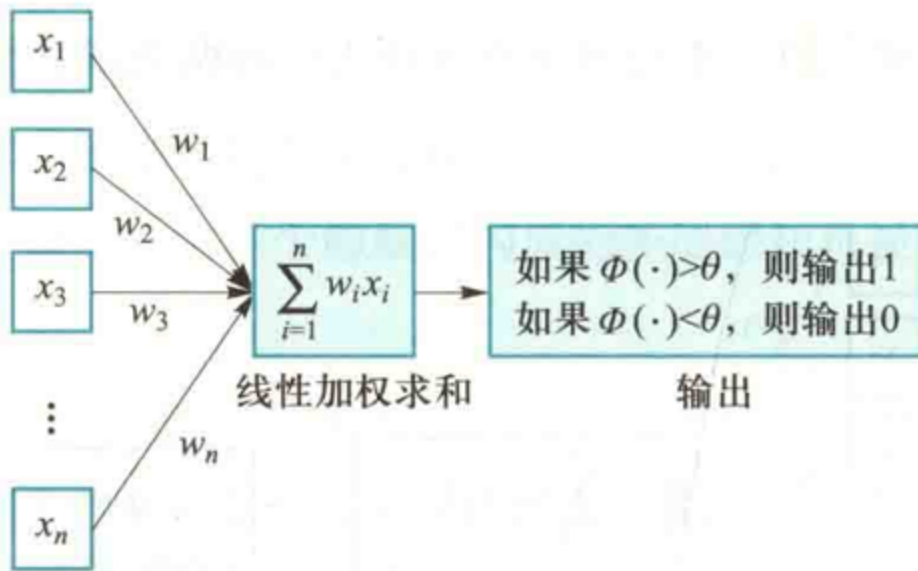


图 5.1 MCP 模型结构

单层感知机

- 输入层：接收实数值的输入向量
- 输出层：输出1或-1

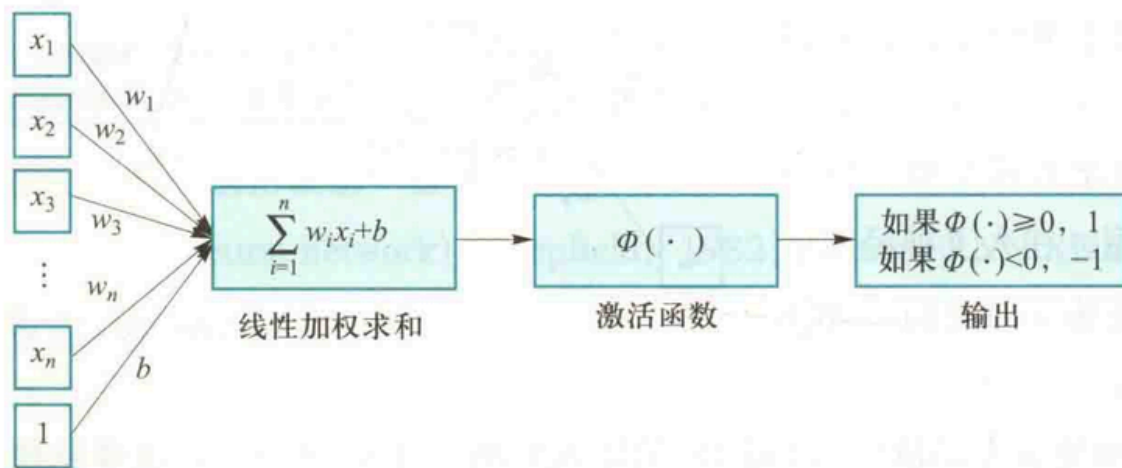


图 5.2 感知机模型

- 单层感知机与MCP模型在连接权重的设置上不同，即感知机中的连接权重参数并不是预先设定好的，而是通过多次迭代训练得到的
- 单层感知机通过构建损失函数来计算模型预测值与数据真实值之间的“误差”，通过最小化损失函数取值来优化模型参数

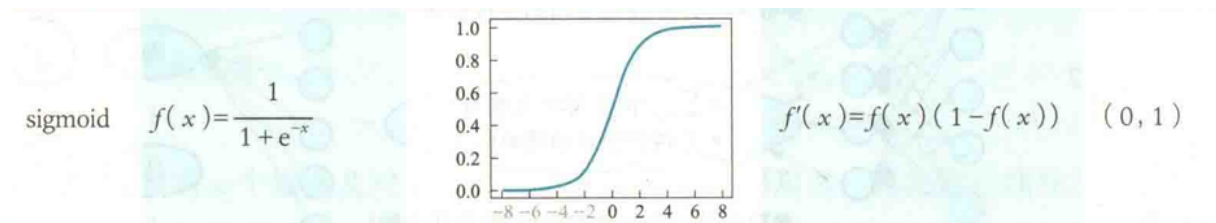
前馈神经网络

- 在单层感知机中增加若干隐藏层，增强非线性表达能力
- 全连接：相邻层之间的神经元相互成对连接，但同层不连
- 每个神经元对前序相邻神经元传递过来的信息按照连接权重加权累加，然后对加权累加结果施以非线性变换
- 神经网络通过亿万个神经元，将输入数据从一个空间映射到另一个空间（**复杂的非线性变化**）
- 映射出现**误差**，则通过**误差反向传播**向前序层传播，逐层更新参数

激活函数

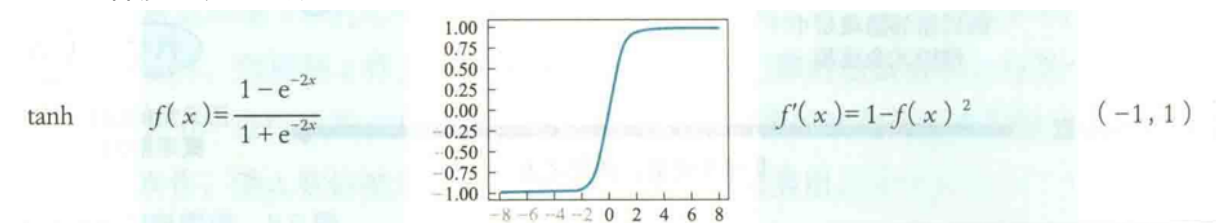
- 实现单个神经元的非线性映射变换
- 为了能够使用梯度下降方法通过误差反向传播来训练神经网络参数，激活函数必须是连续可导的
- sigmoid函数
 - 概率形式输出：值域为(0,1)，可视为概率值
 - 单调递增
 - 非线性变化：x在0附近变化陡峭（容易被激活）
 - 导数小于1，导致在使用反向传播算法更新参数的过程中易出现导数过于接近于0的情

况，即**梯度消失**



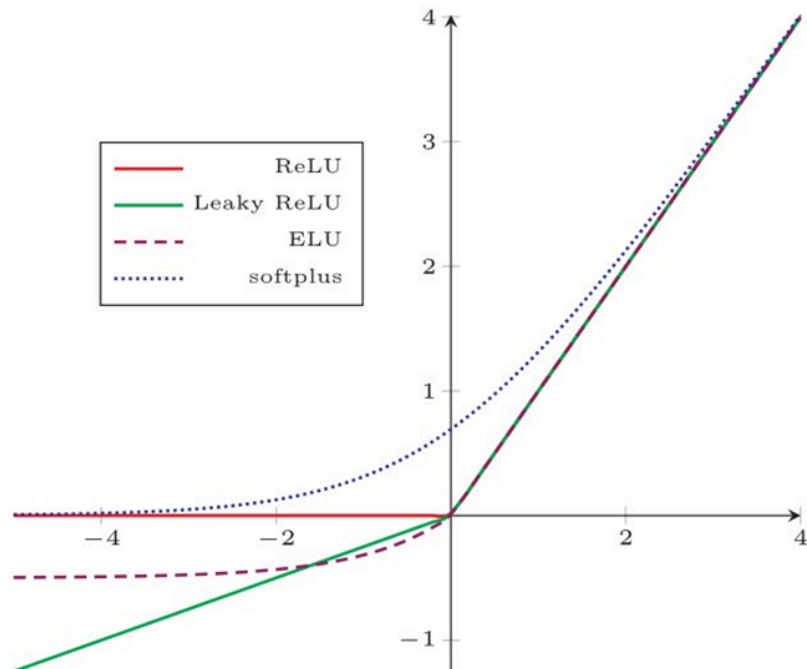
- tanh

- 单调递增，值域(-1,1)，图像和sigmoid相似
- 在0附近梯度比sigmoid大，更容易收敛
- 也存在梯度消失的问题



- ReLU

- 目前普遍使用
- $x \geq 0$ 时，导数为常数，可**缓解梯度消失**问题
- $x < 0$ 时，导数为0，神经元死亡，一定程度上克服过拟合
- 但可能出现大量死亡神经元
 - 可用Leaky ReLU或Parametric ReLU解决，在 $x < 0$ 时梯度非0
 - Leaky为固定参数
 - Parametric为学习参数



- softmax
 - 一般用于多分类任务最后一层
 - 将输入向量 x_i 映射到第 i 个类别的概率 $y_i, 0 < y_i < 1$

$$y_i = \text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^k e^{x_j}}$$

- 输出结果的值累加起来为1，因此可将输出概率最大的作为分类目标

神经网络参数优化

- 是一个**监督学习**的过程
- 通过**损失函数**来计算预测值和真实值之间的误差，来优化参数
- 利用**反向传播算法**，将损失函数所得误差从输出端出发，由后向前传递给每个神经元
- 然后通过**梯度下降算法**对参数进行更新

损失函数

- 均方误差：通过计算误差的平方来衡量模型优劣

$$MSE = \frac{1}{n} \sum_{i=1}^n (\gamma_i - \hat{\gamma}_i)^2$$

- 交叉熵：刻画两个概率分布的之间的距离

$$H(\gamma_i, \hat{\gamma}_i) = -\gamma_i \times \log \hat{\gamma}_i$$

梯度下降

- 损失函数梯度的反方向是损失误差下降最快的方向
- 凸函数具有良好性质：局部最优解即是全局最优解
 - 对于凸函数，梯度下降总能找到函数的全局最小值
 - 损失函数一般采用凸函数
 - sigmoid：负凸正凹
 - tanh：负凸正凹
 - ReLU：全局凸

误差反向传播

- 利用损失函数来计算模型预测结果与真实结果之间的误差
- 从输出端向输入端，由后向前递进进行
- 损失函数梯度反方向是损失误差下降最快的方向
- 求取损失函数梯度时需要各个变量求偏导，这一求取偏导的过程要用到链式法则
 - 求损失函数对于参数的偏导，然后按梯度的反方向选取一个微小增量来调整这个参数的取值
- 假设损失函数为 $L(\theta)$ ，可使用如下梯度更新算法对 θ 进行更新：

$$\theta' = \theta - \eta \times \frac{\partial L}{\partial \theta}$$

- η 为学习速率，影响模型参数的收敛速度
- 链式求导过程可以视为误差反向传播的过程
- 批量梯度下降：在整个训练集上计算损失误差
 - 数据集较大时可能因内存不足而无法完成梯度计算
- 随机梯度下降：使用训练集中的每个训练样本计算损失误差
 - 每次迭代只采用一个样本，收敛速度较慢
 - 但有助于跳出局部最优解
- 小批量梯度下降：二者结合，每次选取一部分样本
 - 通过累加误差来更新参数，训练过程更稳定

- 利用矩阵计算优势

卷积神经网络CNN

对于图像这样的数据，如果将像素点向量与前馈神经网络中的每个神经元相连，则会使模型参数数量变得非常巨大，无法训练

卷积计算

- 卷积核：一个 $m*n$ 的权重矩阵
- 滤波：数据（如图像的像素点）的一块 $m*n$ 区域与卷积核进行加权（相乘）累加
- 特征图：将卷积核在数据上进行扫描，得到的所有卷积滤波结果组合成为特征图
- 无法以数据边缘的点为中心滤波
 - 可以做padding（填充）后再滤波
- 卷积对数据进行了下采样操作（分辨率降低），数据被“约减”，再对被卷积的数据做卷积，如此重复，原始数据中的重要信息就会显现出来
- 卷积核中的系数不是人为确定的，而是通过数据驱动机制学习得到的
- 填充：为了解决无法对边缘位置进行滤波的问题，在边缘周围填充0
 - 这样每个点都可以做中心
 - 也不存在下采样了（分辨率不变）
- 步长：但还是希望分辨率逐步减小
 - 改变卷积核在数据中的移动步长来跳过一些像素
 - 横纵步长可以不同
- 不同卷积核可以用来刻画视觉神经细胞对外界信息感受时的不同选择性
- 感受野：卷积神经网络每一层输出的特征图中的像素点在输入图像中映射的区域大小
- “局部感知、参数共享”的特点减少了网络参数，一定程度上可以防止过拟合
- 被卷积图像大小为 $w*w$ ，卷积核大小为 $F*F$ ，填充行列数为 $P=F/2$ （上取整），步长为 S ，则特征图分辨率为 $(w-F+2P)/S+1$

池化

由于图像中存在较多冗余，在图像处理中，可用**某一区域子块**的统计信息来刻画**该区域中所有像素点**呈现的空间分布模式，以替代区域子块中所有像素点的取值

- 最大池化：选取最大值来代表
- 平均池化：选取平均值来代表

- k-max池化：选择前k大的值来代表

池化操作是一种**下采样**操作

- 对信息进行**约减和抽象**
- **简化**卷积神经网络**计算的复杂度**
- **保持**特征的某种**不变性**(选择、平移和伸缩等)
- 以增强神经网络所学习特征的**稳健性**(防止过拟合与过学习)

全连接

卷积和池化可以捕获图像的局部特征，但无法明了全局特征

使用全连接层可以将不同的局部特征进行组合，完成对全局特征的挖掘（可等效于1*1的卷积运算）

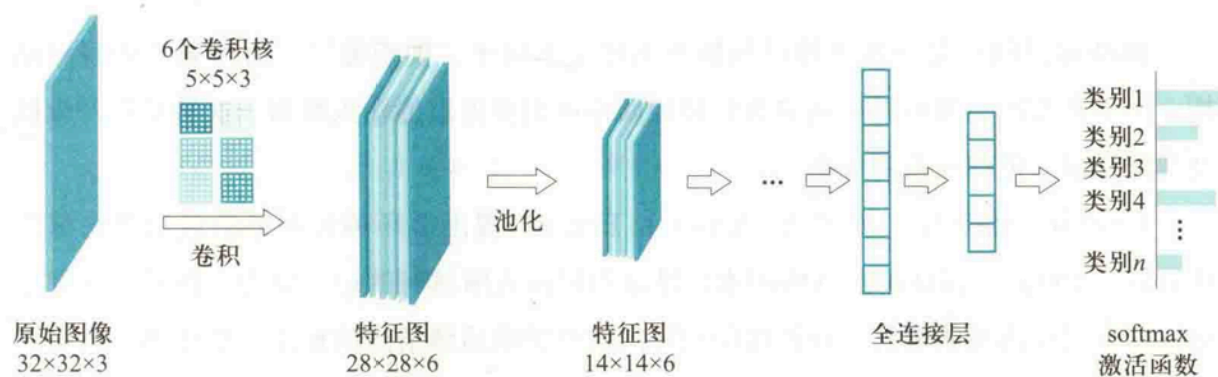


图 5.14 基于卷积神经网络的图像分类示意

循环神经网络RNN

用于处理序列数据（存在前后依赖关系）

在学习过程中记住已经出现的信息，并利用记住的信息影响后续节点的输出

循环神经网络模型

在每一时刻，循环神经网络单元会：

- 输入： x_t, h_{t-1}
 - h_{t-1} 记忆了 t 时刻之前的时序信息
- 输出： $h_t = \Phi(U \times x_t + W \times h_{t-1})$
 - Φ 为激活函数 (sigmoid/tanh)

- U, W 为模型参数

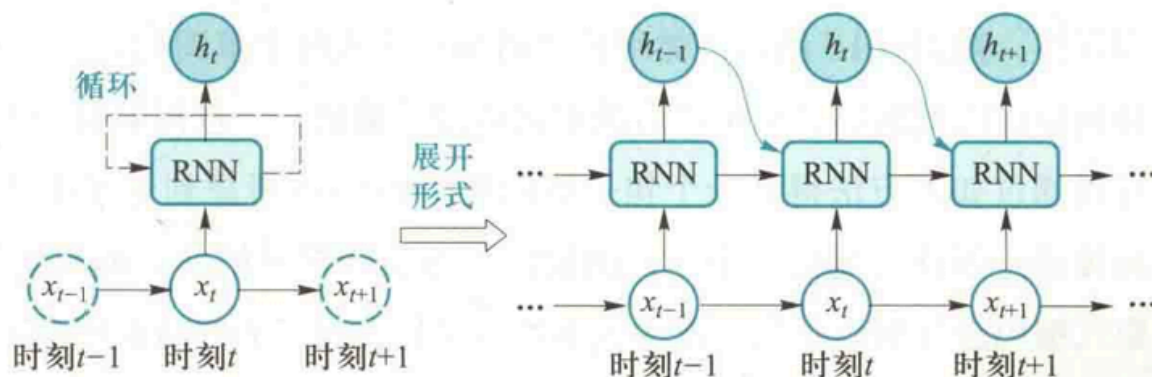


图 5.15 循环神经网络结构及其展开形式示意

- 和前馈神经网络类似，可以利用反向误差传播和梯度下降，被称为“沿时间反向传播”
 - 每个时刻都有一个输出，计算损失误差时，需要将所有时刻的损失累加
- 输入输出对应：
 - 多对多：机器翻译
 - 多对一：文本的情感分类
 - 一对多：图像描述生成
- 相关信息在序列中相距较远时：无法捕捉时序依赖关系
- 输入序列过长：容易出现梯度消失/梯度爆炸

长短时记忆网络LSTM

为了缓解梯度消失问题

- 输入：当前数据 x_t 、前一时刻隐式编码 h_{t-1}
- 输入门： $i_t = \text{sigmoid}(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$
- 遗忘门： $f_t = \text{sigmoid}(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$
- 输出门： $o_t = \text{sigmoid}(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$
- 内部记忆单元： $c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$
 - i_t 控制有多少信息 x_t 流入 c_t
 - f_t 控制有多少记忆 h_{t-1} 可累积到 c_t
- 输出： $h_t = o_t \odot \tanh(c_t)$
- $\odot$ 代表向量内积

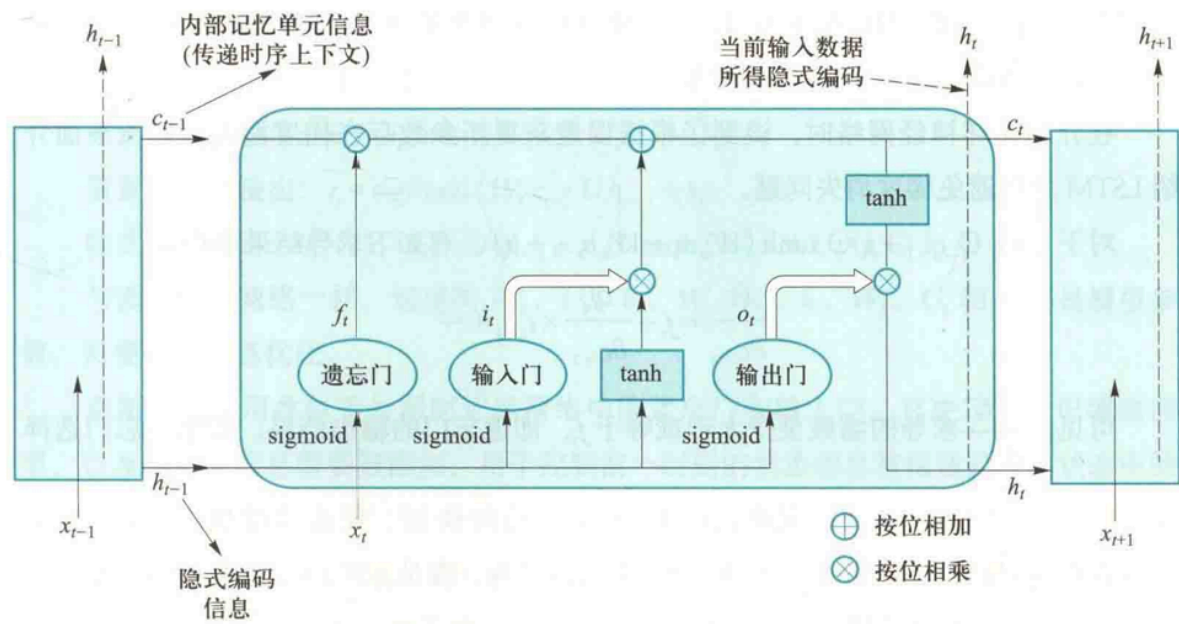


图 5.19 长短时记忆网络模型

避免梯度消失

- 对 c_t 求导有: $\frac{\partial c_t}{\partial c_{t-1}} = f_t + \frac{\partial f_t}{\partial c_{t-1}} + \dots$
- 结果至少大于或等于 f_t , 使得梯度存在
- 通过引入门结构, 在从 t 到 $t+1$ 过程中引入加法来进行信息更新

门控循环单元GRU

对LSTM的简化

- 更新门: $z_t = \text{sigmoid}(W_z x_t + U_z h_{t-1} + b_z)$
 - 相当于遗忘门和输出门, 值越大说明前一时刻的状态信息保留越多
- 重置门: $r_t = \text{sigmoid}(W_r x_t + U_r h_{t-1} + b_r)$
 - 控制忽略前一时刻状态信息的程度, 值小说明越忽略
- 隐式编码: $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$

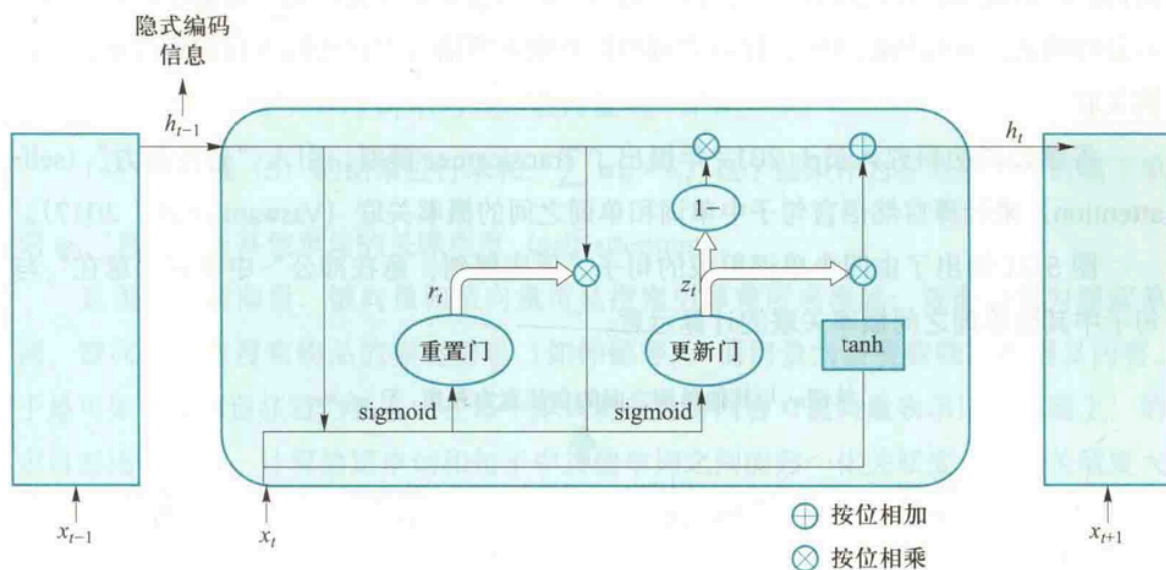


图 5.20 GRU 网络结构

注意力机制

自注意力模型transformer

- 首先生成每个单词的内嵌向量 w_i ，并计算每个单词的：
 - 查询向量： $q_i = W^q \times w_i$
 - 键向量： $k_i = W^k \times w_i$
 - 值向量： $v_i = W^v \times w_i$
 - 这三个映射矩阵对所有单词都一样
- 计算关联度：单词i与单词j之间的关联度为 $\alpha_{ij} = q_i \times k_j$
- 归一化：对单词i与全文所有单词的关联度进行归一化： $\alpha'_{ij} = \text{softmax}(\alpha_{ij})$ ， α'_{ij} 代表单词i与其他单词之间的注意力
- 加权求和：将注意力权重与值向量相乘并求和，得到自注意力结果 $\text{self-attention}(w_i) = \sum_j \alpha'_{ij} \times v_j$

正则化

为了缓解过拟合现象

- Dropout：训练过程中随机丢掉一部分神经元（以一定概率随机屏蔽每一层中的若干神经

元)

2. 批归一化:

- 随深度增加, 输入数据整体分布向值域上下限两端偏移, 导致靠近输入端处容易出现梯度消失
- 把每层中任意神经元的输入值分布改成标准正态分布, 从而输入值被映射到非线性函数梯度较大的区域, 克服梯度消失

3. L1和L2正则化

- 在机器学习中提到, 可以将损失函数加入正则化项: $\frac{1}{n} \sum_{i=1}^n Loss(y_i, f(x_i)) + \lambda \Phi(W)$
- $\Phi(W)$ 为正则化项, 一般用 W 的范数形式表示
 - L0范数: 二分化, 统计不为0的元素的个数
 - L1范数: 各个元素绝对值之和
 - L2范数: 各个元素平方和的开方